

UF6

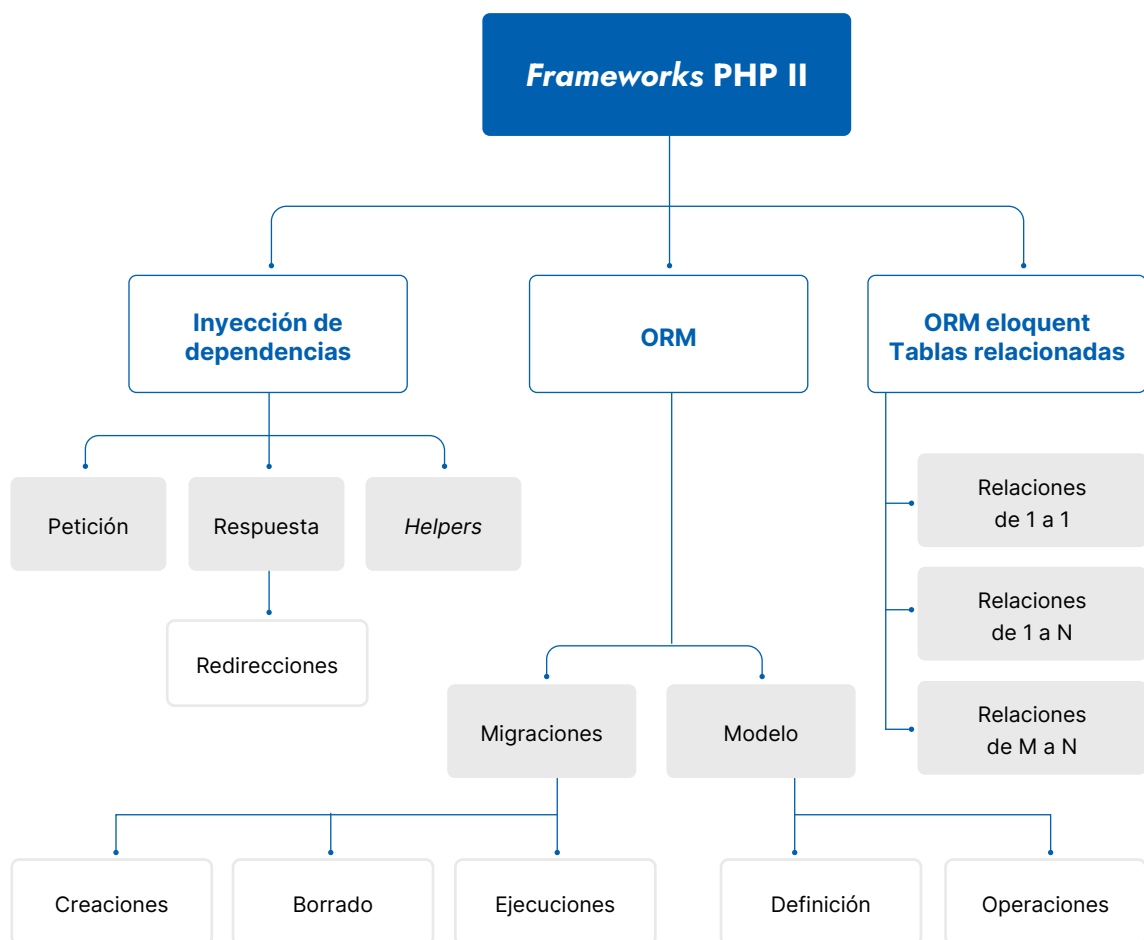
Frameworks para PHP II

Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual	03
1. Inyección de dependencias	04
1.1. Inyección de dependencias en Laravel	04
A. ¿Qué es la inyección de dependencias?	04
B. Contenedor de servicios de Laravel	04
1.2. Ejemplo: la petición del usuario	06
1.3. Ejemplo: la respuesta.....	06
1.4. Ejemplo: helpers	08
2. Mapeo objeto-relacional (ORM)	11
2.1. ¿Qué es un ORM?	11
2.2. Parámetros de conexión a la base de datos.....	12
2.4. Las migraciones	13
A. Estructura de las migraciones	13
B. Creación de migraciones	14
C. Ejecución y borrado de migraciones	15
3. Modelo de datos: operaciones con tablas simples.....	16
3.1. Crear el modelo	16
A. Otras opciones de crear modelos	18
B. Seguir una nomenclatura uniforme	18
3.2. Operaciones sobre el modelo. Primeros pasos con Eloquent.....	19
A. Realizar búsquedas	19
B. Fichas de objetos individuales	20
C. Inserciones	21
D. Modificaciones.....	22
4. Operaciones con tablas relacionadas usando ORM	23

Mapa conceptual



01 Inyección de dependencias

1.1. Inyección de dependencias en Laravel

El concepto de inyección de dependencias es muy habitual en el uso de **frameworks**. Consiste en un mecanismo que facilita recursos a los diferentes componentes de la aplicación, y es algo que ya hemos utilizado, sin saberlo, en los métodos que se han generado para los controladores.

A. ¿Qué es la inyección de dependencias?

La inyección de dependencias (DI) es un término elegante para expresar **«pasar cosas»**. Todo lo que realmente significa es pasar las dependencias de un objeto a través del constructor o métodos, en lugar de crearlos en la creación del objeto dentro del objeto. De esta forma, existirá una única instancia de cada dependencia, que será inyectada en todos los objetos que dependen de ella.

IMPORTANTE

En palabras simples, la inyección de dependencias te permite pasar a través del constructor o métodos de la clase todos los objetos que necesita tu clase para funcionar. En palabras aún más simples: una dependencia es una instancia de la clase B, que la clase A necesita para poder hacer su trabajo.

No es un problema limitado al mundo POO solamente; una **dependencia** es una situación donde el componente A necesita (depende de) el componente B para hacer las cosas que se supone que debe hacer. La palabra también se usa para describir el componente dependiente en este escenario.

B. Contenedor de servicios de Laravel

El contenedor de servicios de Laravel es una herramienta poderosa para administrar dependencias de clases y realizar la inyección de dependencias. La inyección de dependencias esencialmente significa esto: las dependencias de clase se «inyectan» en la clase a través del constructor o, en algunos casos, a través de los métodos o acciones del controlador.

Veamos un ejemplo simple:

```
<?php
namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Repositories\UserRepository;
use App\Models\User;

class UserController extends Controller
{
    /**
     * Implementación del repositorio de usuarios
     *
     * @var UserRepository
     */
    protected $users;

    /**
     * Creamos una instancia de un nuevo controlador.
     *
     * @param UserRepository $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        $this->users = $users;
    }

    /**
     * Mostramos el perfil para el usuario dado.
     *
     * @param int $id
     * @return Response
     */
    public function show($id)
    {
        $user = $this->users->find($id);

        return view('user.profile', ['user' => $user]);
    }
}
```

En este ejemplo, **UserController** necesita recuperar usuarios de una fuente de datos. Por lo tanto, inyectaremos un servicio que pueda recuperar usuarios. En este contexto, es muy probable que nuestro **UserRepository** utilice **Eloquent** para recuperar la información del usuario de la base de datos. Sin embargo, dado que se inyecta el repositorio, podemos intercambiarlo fácilmente con otra implementación. También podemos «simular» fácilmente o crear una implementación ficticia del **UserRepository** al probar nuestra aplicación.

Una comprensión profunda del contenedor de servicios de Laravel es esencial para crear una aplicación potente y grande, así como para contribuir al núcleo de Laravel.

1.2. Ejemplo: la petición del usuario

Por ejemplo, cuando definimos un método en un controlador que necesita procesar una petición, se le pasa como parámetro un objeto del tipo **Request**. Automáticamente, Laravel procesa el tipo de dato y obtiene el objeto asociado (en este caso, la petición del cliente).

```
class ClienteController extends Controller
{
    public function store(Request $request)
    {
        ...
    }
}
```

1.3. Ejemplo: la respuesta

Al igual que tenemos un objeto **Request** para obtener datos de la petición, también existe un **Response** para gestionar la respuesta. Laravel proporciona un método **response**, al que le podemos pasar varios parámetros:

1

El contenido de la respuesta.

2

El código de estado HTTP de respuesta (si no se especifica, es 200, de forma predeterminada).

3

Un *array* con las cabeceras de respuesta (de forma predeterminada, está vacío).

Así, si queremos emitir una respuesta determinada con su código de estado HTTP desde un controlador, podemos hacer esto (por ejemplo, para un código 201):

```
response("Mensaje de respuesta", 201);
```

Las cabeceras pueden especificarse como un *array*, o enlazando llamadas al método **header** (una para cada cabecera):

```
response("Mensaje de respuesta", 201)
->header('Cabecera1', 'Valor1')
->header('Cabecera2', 'Valor2');
```

En el caso de querer devolver un objeto como respuesta, podemos emplear el método **json** de la respuesta (más adelante, veremos que todos los objetos emitidos directamente al cliente se envían en formato JSON), y así podremos adjuntar un código de estado diferente de 200:

```
return response()->json(['datos' => datos], 201)
->header('Cabecera1', 'Valor1');
```

Usar la respuesta para hacer redirecciones

Existe también un método **redirect** que podemos emplear para redireccionar a una ruta desde otra, bien especificando la ruta como parámetro...

```
redirect('/');
```

... o bien indicando una ruta con nombre:

```
redirect()->route('inicio');
```

Podemos pasar valores a la siguiente redirección, almacenándolos en sesión con el método **with**, aunque estos valores se perderán en la siguiente petición (no se quedan almacenados en sesión):

```
redirect()->route('inicio')->with('mensaje', 'Mensaje enviado
correctamente');
```

Para acceder a este mensaje desde la vista afectada, debemos utilizar la función **session**:

```
@if(session()->has('mensaje'))
{{ session('mensaje') }}
@endif
```

Por último, notemos que si hacemos la redirección desde dentro de un método de un controlador (por ejemplo, para redirigir a una ruta desde otra), deberemos devolver (**return**) el resultado de esa redirección para que surta efecto:

```
return redirect()->route('libros.index');
```

1.4. Ejemplo: *helpers*

Para terminar esta introducción a lo que supone la inyección de dependencias en *frameworks* de desarrollo, vamos a hacer uso de una herramienta muy útil en algunas situaciones: los **helpers**.

Un **helper** es básicamente una función de utilidad que podemos querer utilizar en diversos puntos de nuestra web, y que necesitamos tener localizada y compartida.

Uno de los **helpers** más importantes es **The Global Session Helper**.

Hay dos formas principales de trabajar con datos de sesión en Laravel: el **helper** global de sesión **The Global Session Helper** y a través de una instancia **Request**. En primer lugar, vamos a ver el acceso a la sesión a través de una instancia de **Request**, que puede ser de **tipo-hinted** en un cierre de ruta o método de controlador. Recuerda que las dependencias de los métodos del controlador se inyectan automáticamente a través del contenedor de servicios de Laravel.

A través de una instancia de **Request**:

```
$value = $request->session()->get('key');
```

A través del helper global de sesión:

```
Route::get('/home', function () {  
    // Obteniendo un dato de una sesión...  
    $value = session('key');  
  
    // Especificando un valor por defecto...  
    $value = session('key', 'default');  
  
    // Almacenar un dato en la sesión...  
    session(['key' => 'value']);  
});
```

Obtener todos los datos de una sesión:

```
$data = $request->session()->all();
```

Para almacenar una variable de sesión, se puede usar el método **put** de la instancia **request** o el **helper global de sesión**:

```
// Vía una instancia request...
$request->session()->put('key', 'value');

// Vía the global "session" helper...
session(['key' => 'value']);
```

Por ejemplo, imaginemos que queremos resaltar en nuestro menú de navegación la opción que tenemos actualmente visible. Veamos un ejemplo de **helper**, para realizar una función que nos resalte el menú con algún estilo determinado. Para ello, podemos definir una clase CSS con el estilo que queramos para resaltar (esto lo haremos aparte, en los archivos CSS del proyecto), y después utilizar esa clase CSS en una condición para cada menú de navegación.

Por ejemplo, supongamos que la clase CSS para identificar el menú activo se llama **activo**. En este caso, para un menú de varias opciones como este, basta con utilizar el método **routeIs** de la petición (**request**) para comprobar si la ruta coincide con cada menú, y mostrarlo como activo o no, usando un operador ternario de comparación:

```
<nav>
  <ul>
    <li class="{{ request()->routeIs('inicio') ? 'activo' : '' }}">
      <a href="/">Inicio</a>
    </li>
    <li class="{{ request()->routeIs('contacto') ? 'activo' : '' }}">
      <a href="/contacto">Contacto</a>
    </li>
    ...
  </ul>
</nav>
```

Esta característica también funciona si las rutas tienen parámetros.

Podemos, en cambio, sacar fuera de la vista la lógica de establecer un campo como activo o no. Para ello, creamos un archivo de utilidad o **helper**. Lo podemos llamar **helpers.php**, y ubicarlo en la misma carpeta **app**. Dentro, definimos la función que nos va a devolver si una ruta está activa o no, a partir de su nombre:

```
function setActive($nombreRuta)
{
  return request()->routeIs($nombreRuta) ? 'activo' : '';
}
```

Y de este modo, nuestra vista simplemente se dedica a llamar a esta función para cada elemento del menú:

```

<nav>
<ul>
<li class="{{ setActive('inicio') }}">
<a href="/">Inicio</a>
</li>
<li class="{{ setActive('contacto') }}">
<a href="/contacto">Contacto</a>
</li>
...
</ul>
</nav>

```

En el caso de querer mantener el enlace activo para cualquier subruta a partir de la original (por ejemplo, cuando estamos viendo la ficha de un registro a partir del listado general, podemos utilizar la *wildcard* de asterisco *):

```

<li class="{{ setActive('peliculas.*') }}">
<a href="{{ route('peliculas') }}">Películas</a>
</li>

```

Sin embargo, para que Laravel cargue el archivo **helpers.php** que acabamos de crear, como no es una clase, debemos indicarlo explícitamente (Laravel carga automáticamente todas las clases de la carpeta **app**, pero no archivos sueltos que no sean clases). Para ello, debemos ir al archivo **composer.json** de la raíz de nuestro proyecto, a la sección **autoload**, y añadir una sección **files** con un *array* con los archivos que queramos que se carguen también:

```

"autoload": {
    "psr-4": {
        "App\\": "app/"
    },
    "files": ["app/helpers.php"]
},

```

Tras efectuar el cambio, debemos decirle a **composer** que vuelva a compilar el autocargador. Desde la carpeta del proyecto, ejecutamos este comando:

```
composer dump-autoload
```

02 Mapeo objeto-relacional (ORM)

2.1. ¿Qué es un ORM?

Un ORM, en inglés *object-relational mapping*, es un modelo de programación que permite mapear las estructuras de una base de datos relacional (SQL Server, Oracle, MySQL, etc.), en adelante RDBMS (*relational database management system*), sobre una estructura lógica de entidades, con el objeto de simplificar y acelerar el desarrollo de nuestras aplicaciones. Las estructuras de la base de datos relacional quedan vinculadas con las entidades lógicas o la base de datos virtual definida en el ORM, de tal modo que las acciones **CRUD** (**create**, **read**, **update**, **delete**) que se van a ejecutar sobre la base de datos física se realizan de forma indirecta por medio del ORM.

La consecuencia más directa es que, además de «mapear», los ORM tienden a «liberarnos» de la escritura o generación manual de código SQL (*structured query language*) necesario para realizar las *queries* o consultas y gestionar la persistencia de datos en el RDBMS.

Así, los objetos o entidades de la base de datos virtual creada en nuestro ORM podrán ser manipulados por medio de algún lenguaje de nuestro interés según el tipo de ORM utilizado; por ejemplo, en nuestro caso, para Laravel utilizaremos **Eloquent**. La interacción con el RDBMS quedará delegada en los métodos de actualización correspondientes proporcionados por el ORM. Los ORM más completos ofrecen servicios para persistir todos los cambios en los estados de las entidades, previo seguimiento o *tracking* automático, sin escribir una sola línea de SQL.

EJEMPLOS DE ORM MÁS UTILIZADOS EN EL AÑO 2022

PHP/Laravel	Eloquent
PHP/Symfony	Doctrine
JAVA/Spring	Hibernate
.NET	Microsoft Entity Framework 6.0
NodeJs	Sequelize

Tabla 1.

Un ORM nos permite olvidarnos del lenguaje SQL. Eloquent es el ORM que incluye Laravel para gestionar de una forma fácil y sencilla los procesos correspondientes al manejo de bases de datos en nuestro proyecto. Gracias a las funciones que provee, podremos realizar complejas consultas y peticiones de base de datos sin escribir una sola línea de código SQL.

En este apartado comenzaremos a ver algunas cuestiones importantes sobre cómo gestiona Laravel el acceso a bases de datos, y qué mecanismos ofrece para sincronizar los datos de nuestra aplicación con los documentos o registros de una base de datos, así como para generar automáticamente la estructura de tablas y campos de la base de datos a partir del modelo de la aplicación.

2.2. Parámetros de conexión a la base de datos

Una de las primeras cosas que debemos hacer para configurar el acceso a la base de datos en nuestro proyecto es establecer los parámetros con los que conectar a dicha base de datos: nombre del servidor, usuario, contraseña, etcétera. Estos parámetros se definen en el archivo `.env`. Dentro de este archivo, debemos modificar las siguientes variables de entorno:

<code>DB_CONNECTION</code>	Tipo de SGBD que se va a usar.
<code>DB_HOST</code>	Dirección o IP del SGBD (127.0.0.1 para conexión local).
<code>DB_PORT</code>	Puerto por el que el SGBD estará escuchando. Por ejemplo, el puerto predeterminado para MySQL es 3306.
<code>DB_DATABASE</code>	Nombre de la base de datos a la que conectar.
<code>DB_USERNAME</code>	Login del usuario para conectar.
<code>DB_PASSWORD</code>	Password del usuario para conectar.

Tabla 2.

En el archivo `config/database.php` existen unos valores por defecto asociados a cada parámetro de configuración del archivo `.env`, de modo que, si no se encuentra el parámetro, se toma el valor predeterminado. Por ejemplo, el SGBD seleccionado si no se especifica ninguno es `mysql`, a juzgar por esta línea del archivo `database.php`:

```
'default' => env('DB_CONNECTION', 'mysql'),
```

2.4. Las migraciones

Las migraciones suponen una especie de control de versiones para una base de datos, y permiten crear y modificar el esquema de dicha base de datos fácilmente.

A. Estructura de las migraciones

Por defecto, Laravel trae unas migraciones predefinidas, que se hallan en la carpeta **database/migrations**. Cada una tiene un nombre de archivo que comienza por la fecha en que se creó, seguida de una breve descripción de lo que contiene (creación de la tabla de usuarios, reseteo de contraseñas...). Puede que algunas de estas migraciones no nos vayan a ser necesarias, con lo que podemos borrarlas directamente, y puede que otras (en especial, la creación de la tabla de usuarios) sí nos sirvan, pero con otros campos, por lo que deberemos editarlas, como veremos a continuación.

Si examinamos el contenido de una migración, todas deben tener dos métodos:

up: permite agregar tablas, columnas o índices a la base de datos.

down: revierte lo hecho por el método anterior.

Si observamos el contenido de un método **up** de los que vienen predefinidos para crear una tabla, vemos que se utilizan distintos métodos para definir los tipos de datos de cada campo de la tabla, como, por ejemplo, **id()**, para campos que puedan contener enteros autoincrementales, o **string()**, para campos de tipo texto. Además, existen otros métodos modificadores para agregar propiedades adicionales, como, por ejemplo, **unique()**, para indicar valores únicos (claves alternativas), o **nullable()**, para indicar que un campo admite nulos. Aquí tenemos un ejemplo de método **up**:

```
public function up()
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });
}
```

De forma predeterminada, como vemos en los ejemplos que se proporcionan, los esquemas se crean con un **id** autonumérico, y unos *timestamps* para indicar la fecha de creación y de modificación de cada registro, que Laravel gestiona de forma automática cuando insertamos o actualizamos contenidos, lo cual resulta muy útil.

Sobre esta base, podemos añadir o quitar los campos que queramos. Para ver los tipos disponibles para las columnas de la tabla, podemos visitar la documentación de Laravel sobre migraciones, en concreto: <https://laravel.com/docs/9.x/migrations>. Conviene tener presente que el tipo `string` que hemos utilizado en el ejemplo anterior tiene una limitación de 255 caracteres. Para textos más grandes, se puede emplear el tipo `text` (20000 caracteres, aproximadamente) o `longText`.

Podemos especificar una clave primaria con el método `primary`, al que le podemos pasar, o bien el nombre del campo clave, o un `array` de campos clave, en el caso de que esta sea compuesta. De forma predeterminada, los campos de tipo `id` se autoestablecen como claves primarias.

```
$table->primary(['campo1', 'campo2']);
```

B. Creación de migraciones

Creamos migraciones con el comando

```
php artisan make:migration nombre_migracion
```

Por ejemplo, para la tabla prueba:

```
php artisan make:migration crear_tabla_prueba
```

Reparemos en que Laravel ya asigna automáticamente la fecha de la migración; solo debemos especificar el nombre descriptivo de esta. Además, si Laravel detecta la palabra `create` en el nombre de la migración, finalizada en `table`, entiende que es para crear una tabla nueva. En cambio, si detecta la palabra `to` (entre otras), y al final la palabra `table`, entiende que se va a alterar o modificar una tabla existente. Esto es gracias a la clase `TableGuesser` incorporada en Laravel, que detecta ciertos patrones en los nombres de migraciones. La diferencia entre la creación y la modificación es que en el método `up` de la migración se utilizará `Schema::create` o `Schema::table` sobre la tabla en cuestión, respectivamente.

En cualquier caso, también podemos especificar un parámetro adicional en el comando de migración para indicar si queremos crear o modificar una tabla, y de este modo podemos definir el nombre de la migración en el idioma que queramos, y sin restricciones de patrones. Estas dos migraciones crean una tabla (pedidos) y modifican otra (usuarios), respectivamente:

```
php artisan make:migration crear_tabla_pedidos --create=pedidos
php artisan make:migration nuevo_campo_usuario --table=usuarios
```

En el caso de la segunda migración, si, por ejemplo, queremos añadir una columna con el número de teléfono de los usuarios, puede quedar así (tanto el método `up` como el `down`):

```
public function up()
{
    Schema::table('usuarios', function(Blueprint $tabla) {
        $tabla->string('telefono')->nullable();
    });
}

public function down()
{
    Schema::table('usuarios', function(Blueprint $tabla) {
        $tabla->dropColumn('telefono');
    });
}
```

Si queremos que el campo en cuestión esté en un orden concreto, podemos usar el método **after** para indicar detrás de qué campo queremos ponerlo (en el método **up**):

```
$tabla->string('telefono')->after('email')->nullable();
```

C. Ejecución y borrado de migraciones

Para ejecutar las migraciones (el método **up** de cada una), lanzamos el siguiente comando desde la carpeta de nuestro proyecto (habiendo creado la base de datos ya previamente, y modificado las credenciales de acceso en el **archivo .env**):

```
php artisan migrate
```

Adicionalmente a las tablas afectadas, se tendrá otra tabla **migrations** en la base de datos con un histórico de las migraciones realizadas. Para cada una, se almacena su **id** (autonumérico), el nombre de la migración y el número de proceso por lotes en que se hizo (aquellas que compartan el mismo número se hicieron a la vez en el mismo lote). De este modo, aquellas que ya se hayan hecho no se volverán a realizar.

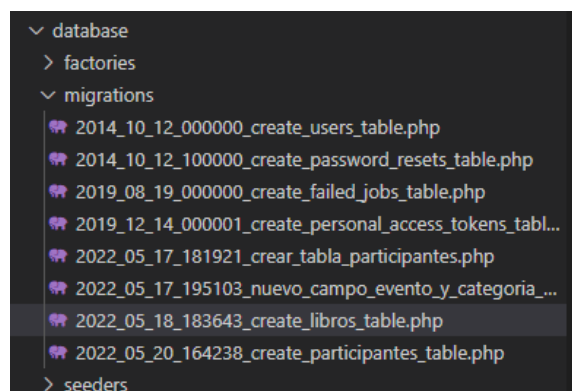


Figura 1.

Para deshacer las migraciones realizadas (ejecutar el método **down** de estas), ejecutamos

```
php artisan migrate:rollback
```

Esto eliminará **todas** las migraciones del último lote existente en la tabla **migrations**. Si no queremos deshacerlo todo, sino retroceder un número determinado de migraciones dentro de ese lote, ejecutamos el comando anterior con un parámetro **--step**, indicando el número de pasos o migraciones que hay que deshacer (en orden cronológico, de más reciente a más antigua):

```
php artisan migrate:rollback --step=2
```

Si volvemos a hacer la migración, se restablecerán las migraciones deshechas de ese lote. Otro comando también muy utilizado es **migrate:fresh**. Lo que hace es eliminar todas las migraciones realizadas y volverlas a lanzar. Es útil cuando, estando en desarrollo, añadimos campos nuevos a alguna tabla y queremos rehacer las tablas completamente.

```
php artisan migrate:fresh
```

IMPORTANTE

El comando **migrate:fresh** es **DESTRUCTIVO** (elimina los contenidos de las tablas) y solo debe utilizarse en entornos de desarrollo, no de producción.

03 Modelo de datos: operaciones con tablas simples

Ahora que ya tenemos la estructura de tablas creada en la base de datos, vamos a ver qué mecanismos ofrece Laravel para acceder a estos datos de forma sencilla desde la aplicación. Veremos cómo definir el modelo de datos asociado a cada tabla y cómo manipular estos datos empleando el ORM Eloquent incorporado con Laravel.

3.1. Crear el modelo

La idea es crear una clase por cada tabla que tengamos en nuestra base de datos, para así interactuar con la tabla a través de dicha clase asociada. Para crear esta clase modelo, utilizamos la opción **make:model** del comando **php artisan**. Le pasaremos como parámetro adicional el nombre de la clase que vamos a crear. Por ejemplo, para el caso de una biblioteca, podemos crear así el modelo **Libro**:

```
php artisan make:model Libro
```

Por convención, los modelos se crean con un nombre en singular, empezando por mayúscula, y se ubican en la carpeta `app\Models`. La estructura básica del modelo es algo así:

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Libro extends Model
{
    ...
}
?>
```

En nuestro caso, vamos también a utilizar el modelo de usuario que ya existe en la carpeta `app\Models`, aunque lo renombraremos de `User` a `Usuario`:

```
<? class Usuario extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
    protected $fillable = [
        'name',
        'email',
        'password',
    ];
}
```

IMPORTANTE

Hasta Laravel 7, los modelos se generaban automáticamente en la carpeta `app`, y era necesario moverlos manualmente a una subcarpeta si queríamos estructurar mejor nuestro código, actualizando también el `namespace` correspondiente. A partir de Laravel 8.x, la ubicación en la carpeta `app\Models` se realiza de forma predeterminada.

Automáticamente, se asocia este modelo a una tabla con el mismo nombre, pero en plural y en minúscula, por lo que los modelos anteriores estarían asociados, respectivamente, a unas tablas `libros` y `usuarios` en la base de datos. En caso de que no queramos que sea así, definimos una propiedad `$table` en la clase con el nombre que queramos que tenga la tabla asociada. Por ejemplo:

```
class Libro extends Model
{
    protected $table = 'mislibros';
}
```

A. Otras opciones de crear modelos

El comando anterior, `make:model`, admite unos parámetros adicionales, de forma que se puede crear a la vez el modelo y la migración, y más aún, el modelo, la migración y el controlador asociado. Veamos algunos ejemplos:

<pre>php artisan make: model Libro -m</pre>	El comando anterior crea un modelo Libro en la carpeta app\Models y, además, crea una migración llamada create_libros_table en la carpeta database/migrations, lista para que editemos el método <code>up()</code> y especifiquemos los campos necesarios. Reparemos en que el nombre de la migración añade una «s» al nombre de la tabla automáticamente, a partir del modelo en singular.
<pre>php artisan make: model Libro -mc</pre>	Este otro comando crea lo mismo que el anterior y además, un controlador llamado LibroController en la carpeta app\Http\Controllers. Este controlador está vacío para que añadamos los métodos que consideremos.
<pre>php artisan make: model Libro -mcr</pre>	Esta otra opción crea lo mismo que la anterior, pero el controlador LibroController es en este caso un controlador de recursos, por lo que tiene ya incorporados el conjunto de métodos propios de este tipo de controladores: index, show, etcétera. Podemos también usar la versión extendida de estos parámetros. Por ejemplo:
<pre>php artisan make: model Libro --migration --controller --resource</pre>	En nuestro caso, como hemos ido creando los controladores y migraciones antes que los modelos, no sería necesario dar este paso, pero ahora que ya empezamos a ver cómo funciona y se interrelaciona todo, puede resultar útil emplear este comando para crear de golpe todas las partes implicadas (modelo, migración y controlador).

Tabla 3.

B. Seguir una nomenclatura uniforme

Recuerda que, en sesiones anteriores, hemos comentado la recomendación/necesidad de seguir una nomenclatura uniforme en los modelos, controladores y vistas. Así, para el modelo **Libro** ya tendríamos su controlador asociado **LibroController**, y las vistas se definirían en la subcarpeta **resources/views/libros**, con los nombres correspondientes a cada método del controlador (por ejemplo, **index.blade.php**, o **show.blade.php**).

3.2. Operaciones sobre el modelo. Primeros pasos con Eloquent

Eloquent es el ORM incorporado por defecto en Laravel. Un ORM (*object-relational mapping*) es una herramienta que permite establecer una relación entre los registros de una tabla de la base de datos y los objetos de una clase (de PHP, en nuestro caso), de forma que los datos de la base de datos se convierten a objetos PHP y viceversa. Además, **Eloquent** implementa el patrón *active record*, que añade a las clases métodos como **save**, **update**, **delete**, etcétera, que permiten interactuar con la base de datos para, respectivamente, insertar, modificar o borrar registros asociados a objetos.

A. Realizar búsquedas

Una vez creado el modelo, y aunque esté vacío, ya podemos utilizarlo en los controladores para acceder a los datos. Basta con importar la clase correspondiente (con **use**), y utilizar los métodos que se heredan de **Model**. Por ejemplo, el método **get** permite obtener los registros de la tabla, convertidos a objetos. Así es como obtendríamos todos los libros de la tabla desde un controlador:

```
use App\Models\Libro;
use Illuminate\Http\Request;

class LibroController extends Controller
{
    public function index()
    {
        $libros = Libro::all();
        return view('libros.index', compact('libros'));
    }
}
```

1

Lo que obtenemos es un *array* de objetos, por lo que deberemos acceder a sus propiedades como tales. Por ejemplo, si queremos mostrar los títulos de los libros en una vista Blade, haríamos algo como esto:

```
@foreach($libros as $libro)
    {{ $libro->título }}
@endforeach
```

2

3

Como alternativa, también podemos obtener una consulta filtrada, especificando con el método **where** la condición que deben cumplir los registros que se quieren obtener. Por ejemplo, así obtendríamos libros con precio inferior a 10 euros y superior a 5 euros:

```
class LibroController extends Controller
{
    public function index()
    {
        $libros = Libro::where('precio', '<', 10)
            ->where('precio', '>', 5)->get();
        return view('libros.index', compact('libros'));
    }
}
```

4

Sobre estas consultas base, podemos aplicar una serie de añadidos. Por ejemplo, podemos querer ordenar los libros por título, para lo que haríamos esto en el controlador:

```
$libros = Libro::orderBy('titulo')->get();
```

5

El método `orderBy` admite un segundo parámetro, que indica el sentido de la ordenación. De forma predeterminada, es ASC (ascendente), pero también puede ser DESC:

```
$libros = Libro::orderBy('titulo', 'DESC')->get();
```

B. Fichas de objetos individuales

1

Una operación bastante habitual es mostrar una ficha de un objeto a partir de un listado, haciendo clic en el título o alguna parte visible de ese objeto. Por ejemplo, si queremos ver los datos de un libro a partir de un listado con sus títulos, podemos hacer algo como esto en la plantilla Blade:

```
@forelse($libros as $libro)
    <li><a href="{{ route('libros.show', $libro) }}">
        {{ $libro->titulo }}
    </a></li>
@endforelse
```

2

Vemos que hemos utilizado el método **route** para indicar la ruta que se ha de seguir, con un segundo parámetro, que en este caso es el objeto concreto de esa fila. Laravel automáticamente lo reemplazará en el enlace por el identificador de dicho objeto. Por su parte, la ruta asociada a este enlace podría ser algo así (en el archivo de rutas):

```
Route::get('/libros/{id}', [LibroController::class, 'show'])
->name('libros.show');
```

3

No obstante, también podemos haber definido las rutas como un paquete de recursos, y cada una tendrá su método asociado:

```
Route::resource('libros', LibroController::class);
```

4

Finalmente, el método `show` del controlador asociado se encargará de obtener los datos del libro a partir de su `id`, y generar la vista correspondiente. Para obtener los datos de un objeto a partir de su identificador, podemos emplear el método `find` del modelo, pasándole como parámetro el identificador. Así, podríamos generar una vista con los datos como esta:

```
public function show(Libro $libro)
{
    //
    $libro = Libro::find($id);
    return view('libros.show', compact('libro'));
}
```

IMPORTANTE

Si devolvemos (`return`) directamente lo que obtiene el método `find`, nos llegará al navegador en formato JSON. De hecho, si devolvemos un *array*, Laravel lo enviará directamente en formato JSON. Esta característica la utilizaremos más adelante para definir servicios REST.

En el caso de que el objeto no se encuentre (porque, por ejemplo, utilicemos un `id` equivocado), la vista generada fallará. Para evitarlo, en lugar del método `find` podemos emplear `findOrFail`, que, en caso de que no se encuentre el objeto, generará una vista con un error 404, más apropiada. Además, recuerda que puedes personalizar estas páginas de error definiendo las vistas correspondientes .

```
$libro = Libro::findOrFail($id);
```

En este punto, y a falta de que podamos hacer inserciones más adelante, puedes probar a insertar unos pocos libros de prueba en la base de datos `biblioteca` desde **phpMyAdmin**, y probar estas dos rutas que hemos hecho (listado y ficha de `libro`).

C. Inserciones

Las inserciones a través de Eloquent se pueden realizar creando una instancia del objeto, rellenando sus atributos y llamando al método `save`, heredado de la superclase `Model`.

```
$libro = new libro();
$libro->titulo = "Programación en Laravel 9.x";
$libro->editorial = "McGrawHill";
$libro->precio = 25;
$libro->save();
```

Como alternativa, también se puede utilizar el método **create** del modelo, y pasarle todos los datos de la petición, que llegarían desde un formulario, como veremos más adelante:

```
public function store(Request $request) {  
    Libro::create($request->all());  
}
```

Para que esto último funcione, deben cumplirse dos premisas:

- » Cada campo de la petición debe tener asociado un campo del mismo nombre en el modelo.
- » Debemos definir en el modelo una propiedad llamada **\$fillable** con los nombres de los campos de la petición que nos interesa procesar (el resto se descartan). Esto es obligatorio especificarlo, aunque nos interesen todos los campos, para evitar inserciones masivas malintencionadas (por ejemplo, editando el código fuente para añadir otros campos y modificar datos inesperados).

```
class Libro extends Model {  
    use HasFactory;  
    protected $fillable = ['titulo', 'editorial', 'precio'];  
}
```

Este código de inserción (o bien campo a campo, o usando el método **all**) se suele poner en el método **store** del controlador, para que reciba los datos del formulario de inserción y la haga en la base de datos.

D. Modificaciones

La modificación consiste en dos pasos:

Encontrar el objeto que se va a modificar (buscándolo por el **id** con **findOrFail**, por ejemplo, como se ha explicado antes) Modificar las propiedades que se necesiten, y llamar al método **save** del objeto para guardar los cambios.

```
$libroAModificar = Libro::findOrFail($id);  
$libroAModificar->titulo="Otro título";  
$libroAModificar->save();
```

También podemos utilizar el método **update** enlazado con **findOrFail**, y pasarle como parámetro todos los datos de la petición, igual que se ha explicado para la inserción, y siempre y cuando hayamos declarado el atributo **\$fillable** en el modelo para indicar qué campos se aceptan:

```
Libro::findOrFail($id)->update($request->all());
```

Este código de modificación se suele poner en el método **update** del controlador, para que reciba los datos del formulario de edición y haga la modificación correspondiente.

Tienes más información sobre paginaciones y borrados en el documento **Operaciones con tablas simples, paginaciones y borrados**, en la sección **Descarga de documentos**.

04 Operaciones con tablas relacionadas usando ORM

En este apartado continuaremos profundizando con lo visto en el anterior en cuanto a mecanismos de acceso a datos desde Laravel. Por un lado, veremos cómo podemos poblar las tablas de nuestra base de datos con una serie de datos ya precargados, e incluso con datos ficticios que nos sirvan para unas pruebas iniciales, que luego se puedan descartar. También veremos qué tipos de relaciones se pueden establecer entre los modelos de la aplicación, y cómo se reflejan automáticamente en la base de datos.

Eloquent permite definir relaciones de varios tipos entre tablas. Estas se definen, como veremos a continuación, a través de los distintos modelos involucrados en la relación.

RELACIONES UNO A UNO O ONE TO ONE

Supongamos que tenemos dos modelos **Usuario** y **Telefono**, de modo que podemos establecer una relación uno a uno entre ellos: un usuario tiene un teléfono, y un teléfono pertenece a un usuario.

Para reflejar esta relación en tablas, una de las dos debería tener una referencia a la otra. En este caso, podríamos tener un campo **usuario_id** en la tabla de **telefonos** que indique a quién pertenece dicho teléfono. Es importante que el campo se llame **usuario_id**, como veremos a continuación.

Para indicar que un usuario tiene un teléfono, añadimos un nuevo método en el modelo de **Usuario**, que se llame igual que el modelo con el que queremos conectar (**telefono**, en este caso):

```
class Usuario extends Model
{
    public function telefono()
    {
        return $this->hasOne(Telefono::class);
    }
}
```

Ahora, si queremos obtener el teléfono de un usuario, basta con que hagamos esto:

```
$telefono = Usuario::findOrFail($id)->telefono;
```

Hemos empleado una característica de **Eloquent** denominada propiedades dinámicas, por la cual podemos referenciar un método de relación como si fuera una propiedad (en lugar de usar `telefono()`, hemos empleado `telefono`).

La instrucción anterior obtiene el objeto `telefono` asociado con el usuario buscado (a través del `id` del usuario). Para que esta asociación tenga efecto, es preciso que en la tabla `telefonos` exista un campo `usuario_id` y que se corresponda con un campo `id` de la tabla de `usuarios`, de modo que **Eloquent** establece la conexión entre una y otra tabla. Debemos definir una nueva migración de modificación sobre la tabla `telefonos` para añadir ese nuevo campo, o refrescar la migración original con él y borrar los contenidos previos.

Si queremos utilizar otros campos distintos a los anteriores en una y otra tabla para conectarlas, debemos indicar dos parámetros más al llamar a `hasOne`. Por ejemplo, así relacionaríamos las dos tablas anteriores, indicando que la clave ajena de `telefonos` es `idUsuario`, y que la clave local a la que se referencia en usuarios es `codigo`:

```
return $this->hasOne(Telefono::class, 'idUsuario', 'codigo');
```

También es posible obtener la relación inversa; es decir, a partir de un teléfono, obtener el usuario al que pertenece. Para ello, añadimos un método en el modelo `Telefono` y empleamos el método `belongsTo` para indicar a qué modelo se asocia:

```
class Telefono extends Model
{
    public function usuario()
    {
        return $this->belongsTo(Usuario::class);
    }
}
```

Nuevamente, podemos especificar otros nombres de clave pasando parámetros adicionales a `belongsTo`, igual que se hace para `hasOne`.

De este modo, si queremos obtener el usuario a partir del teléfono, podemos hacerlo así:

```
$usuario = Telefono::findOrFail($idTelefono)->usuario;
```

Guardar datos relacionados

Supongamos que queremos guardar un usuario con su teléfono asociado.

```
// Buscamos el teléfono que queremos asociar
// (suponiendo que existe previamente)
$telefono = Telefono::findOrFail($idTelefono);
$usuario = new Usuario();
$usuario->nombre = "Pepe";
$usuario->email = "pepe@gmail.com";
//Podemos simplemente guardar el id del teléfono como un campo más del
usuario:
$usuario->telefono_id = $telefono->id;
//O podemos vincular ambos objetos en la relación, usando el método
associate, de este modo
$usuario->save();
```

RELACIONES UNO A MUCHOS O ONE TO MANY

Para ilustrar esta relación, veamos otro ejemplo: supongamos que tenemos los modelos **Autor** y **Libro**, de modo que un autor puede tener varios libros, y un libro está asociado a un autor. La forma de establecer la relación entre ambos consistirá en añadir en la tabla de **libros** una clave ajena al **autor** al que pertenece. A la hora de plasmar esta relación en los modelos, se hace de forma similar al caso anterior, solo que en lugar de utilizar el método **hasOne** en la clase **Autor** usaríamos el método **hasMany**:

```
class Autor extends Model
{
    public function libros()
    {
        return $this->hasMany(Libro::class);
    }
}
```

Igual que ocurría antes, se asume que la tabla de libros tiene una clave primaria **id**, y que la clave ajena correspondiente hacia la tabla de autores es **autor_id**. De lo contrario, se pueden especificar otros pasando más parámetros a **hasMany**.

De este modo, obtenemos los libros asociados a un autor:

```
$libros = Autor::findOrFail($id)->libros;
```

Finalmente, también podemos establecer la relación inversa, y recuperar el **autor** al que pertenece un determinado **libro**, definiendo un método en la clase **Libro** que emplee **belongsTo**, como en las relaciones uno a uno:

```
class Libro extends Model
{
    public function autor()
    {
        return $this->belongsTo(Autor::class);
    }
}
```

Y obtener, por ejemplo, el nombre del autor a partir del libro:

```
$nombreAutor = Libro::findOrFail($id)->autor->nombre;
```

Aplicación de esta relación en nuestro ejemplo

Esta relación la podemos dejar plasmada en nuestro ejemplo de la biblioteca, definiendo un nuevo modelo **Autor** con su correspondiente migración, y relacionando los modelos. Para ello, seguiremos estos pasos:

1. Creamos una nueva migración de modificación sobre la tabla de libros, para añadir un nuevo campo, **autor_id**.

```
php artisan make:migration nuevo_campo_autor_libros --table=libros
```

```
class NuevoCampoAutorLibros extends Migration
{
    public function up()
    {
        Schema::table('libros', function(Blueprint $table) {
            $table->integer('autor_id');
        });
    }
}
```

2. Creamos de golpe el modelo, la migración y el controlador de autores (aunque el controlador no lo vamos a utilizar, al menos por el momento). El modelo **Autor** se crea por defecto en la carpeta **app\Models**, junto con el de usuarios y el de libros.

```
php artisan make:model Autor -mcr
```

Deberemos renombrar a mano la migración, ya que el plural que asignará Laravel de forma predeterminada será **autores**, y no **autores**. Por eso, hemos de cambiar tanto el nombre del fichero de la migración como el nombre de la clase interna y el nombre de la tabla a la que se referencia en los métodos **up** y **down**.

3. Editamos la migración para definir los campos que tendrá la nueva tabla de autores en su método `up`: un nombre y un año de nacimiento (opcional):

```
class CreateAutoresTable extends Migration
{
    public function up()
    {
        Schema::create('autores', function(Blueprint $table) {
            $table->id();
            $table->string('nombre');
            $table->integer('nacimiento')->nullable();
            $table->timestamps();
        });
    }
}

php artisan migrate
```

4. Añadimos en el modelo `Autor` que la tabla asociada será `autores` (de lo contrario, considera que será `autores`). Además, definimos una relación de uno a muchos con los libros, añadiendo el método siguiente:

```
class Autor extends Model
{
    protected $table = 'autores';

    public function libros()
    {
        return $this->hasMany(Libro::class);
    }
}
```

5. Recíprocamente, añadimos al modelo `Libro` este otro método, para poder recuperar un autor a partir de uno de sus libros:

```
class Libro extends Model
{
    ...

    public function autor()
    {
        return $this->belongsTo(Autor::class);
    }
}
```

6. Utilizando **phpMyAdmin**, creamos a mano un par de autores en la tabla de autores, y los relacionamos con algunos de los libros que haya en la tabla de libros, añadiendo también a mano el `id` de cada autor en la clave ajena correspondiente de los libros.
-

7. Para probar cómo funcionan las relaciones, vamos primero a crear un nuevo libro asociado al autor 1. Definimos una ruta de prueba y la ruta de libros en el archivo `routes/web.php` con este código (deberemos incorporar con `use` los modelos de `Autor` y `Libro`):

```
Route::get('/relacionPrueba', function() {
    $autor = Autor::findOrFail(1);
    $libro = new Libro();
    $libro->titulo = "Libro de prueba " . rand();
    $libro->editorial = "Editorial de prueba";
    $libro->precio = 5;
    $libro->autor()->associate($autor);
    $libro->save();
    return redirect()->route('libros.index');
});

Route::resource('libros', 'LibroController');
```

8. Ahora, modificamos la vista `libros/index.blade.php` para que, en el listado, utilice las relaciones entre tablas para mostrar el nombre del autor entre paréntesis junto al título de cada libro:

```
@foreach($libros as $libro)
    <li><a href="{{ route('libros.show', $libro) }}">
        {{ $libro->titulo }} ({{ $libro->autor->nombre }})
    </a></li>
@endforeach
@empty
    <li>No se encontraron libros</li>
@endforeach
```

9. Podemos probar las dos cosas accediendo respectivamente a estas dos URL (suponiendo que el servidor está escuchando en localhost por el puerto 8000):

```
http://localhost:8000/relacionPrueba
http://localhost:8000/libros
```

Acceso eficiente a datos relacionados. *Eager loading*

En el ejemplo anterior, hemos visto cómo podemos obtener, dado un libro, el nombre del autor de este modo en una vista Blade:

```
{{ $libro->autor->nombre }}
```

Sin embargo, este código provoca una nueva consulta en la base de datos para buscar los datos del autor asociado al libro, con lo que, para un listado de 100 libros, estaremos haciendo 100 consultas adicionales para extraer la información de los respectivos autores.

Para evitar esta sobrecarga, podemos emplear una técnica llamada **eager loading** (que en español podríamos traducir como «carga apresurada o impaciente»). Consiste en emplear el método `with` para indicar qué relación queremos dejar precargada en el resultado. Por ejemplo, si indicamos algo así en la función `index` del controlador de libros, recuperará tanto el autor como los libros asociados en una sola consulta a la base de datos.

```
public function index()
{
    $libros = Libro::with('autor')->get();
    return view('libros.index', compact('libros'));
}
```

Tabla 5.

RELACIONES MUCHOS A MUCHOS O MANY TO MANY

Estas relaciones son más difíciles de plasmar, ya que es necesario contar con una tercera tabla que relacione las dos tablas afectadas. Pero vayamos por partes...

Para ilustrar este caso, supongamos los modelos `Usuario` y `Rol`, de modo que un usuario puede tener varios roles, y un rol puede ser asignado a varios usuarios. Nuevamente, definimos un método en el modelo `Usuario` que utilice el método `belongsToMany` para indicar con qué otro modelo se relaciona:

```
class Usuario extends Model
{
    public function roles()
    {
        return $this->belongsToMany(Rol::class);
    }
}
```

Así ya podremos acceder a los roles de un usuario:

```
$roles = Usuario::findOrFail($id)->roles;
```

En este caso, al otro lado de la relación hacemos lo mismo: definimos un método en el modelo `Rol` que indique con `belongsToMany` que puede pertenecer a varios usuarios:

```
class Rol extends Model
{
    public function usuarios()
    {
        return $this->belongsToMany(Usuario::class);
    }
}
```

A efectos de automatización, es decir, para que Eloquent establezca los nexos de forma automática, si queremos establecer una relación muchos a muchos entre un modelo A y otro B, se asume que existirá otra tabla `a_b` (el orden en que se colocan los nombres de las tablas es alfabético), con los campos `a_id` y `b_id`, que relacionen los dos modelos. En nuestro caso, se asumirá que existe una tabla `rol_usuario` con un campo `rol_id` y otro llamado `usuario_id`, que enlacen con los correspondientes `id` de las tablas de usuarios y roles. Si esto no fuera así, podemos pasar más parámetros a `belongsToMany` para indicarlo.

En el caso de las relaciones muchos a muchos, es posible que nos interese acceder a algún dato de esa tabla intermedia que los relaciona. En ese caso, hacemos uso del atributo `pivot`, predefinido, y que apunta a la tabla o modelo intermedio entre los dos relacionados. Por ejemplo, si quisiéramos obtener la fecha de creación de la relación entre un usuario y un rol, podríamos hacer esto:

```
$roles = Usuario::findOrFail($id)->roles;
for($roles as $rol)
{
    echo $rol->pivot->created_at;
}
```

Tabla 6



VÍDEOLAB

Para consultar el siguiente vídeo sobre página de productos, escanea el código QR o [pulsa aquí](#).

UAX FP